

Docket

A clear record of what was checked.

The problem

Small teams often copy crypto payment links into a spreadsheet. The next reviewer cannot see whether the call succeeded, whether the payment matched an invoice, or whether anyone checked the underlying evidence.

The workflow

Set the expectation. Save the invoice reference, payee, expected wallet and exact amount beside an Ethereum Sepolia transaction.

Check the source. Read the receipt. Compare direct ETH transfers or the selected token's Transfer logs with the expected recipient and amount.

Verify inclusion. Use Attestcoin to prove that the source transaction is included in an attested block, through Creditcoin's native verifier.

Leave a usable record. Save a review note and export public evidence. Share the private invoice bundle only with the reviewer who needs it.

Why Creditcoin and Attestcoin matter

A normal database can store an explorer link and a note. Attestcoin adds a verifiable source transaction and continuity proof. Docket checks that proof on CC3 and binds the proven source position to a salted invoice commitment in its registry contract.

Privacy and scope

Invoice details and random salts stay in local SQLite storage. The public export omits private text and notes. Docket moves no funds, creates no credit score, and does not prove that an off-chain invoice is genuine or owned by its creator.

Repository: github.com/gylshaurya/docket

How the evidence is checked

Built and verified behavior, with remaining work kept explicit.

Integration path

The pinned Gluwa USC SDK fetches the source transaction and receipt, encodes them in the sponsor's EVM v1 format, requests a ProofBuilder proof, and checks chain key, height, index, hash and encoded bytes. It then calls the native BlockProver on CC3 testnet.

DocketRegistry stores the submitter, invoice commitment, encoded transaction digest and verified source position. Duplicate keys are scoped to each submitter so another wallet cannot front-run and reserve their record. Other wallets can make their own claims, which is not invoice ownership proof.

Verified during development

A real public Sepolia transaction passed source receipt and transfer checks, then passed an actual read-only CC3 inclusion call. A second native call rejected altered proof bytes. The invoice expectation was clearly labelled illustrative; no customer payment claim was made.

Seven storage tests, four SDK boundary tests and twelve contract tests passed. The contract tests include two 256-case fuzz checks and an explicit test verifier. Browser checks covered review notes, draft retention, inline errors, public/private exports and desktop/mobile layout. The design finish review resolved all four findings.

What is still pending

Public CC3 registry deployment and a confirmed registry write need free testnet gas. Public judge access, the recorded video and final submission approval remain pending. Read-only verification is not a deployed registry. The local role is not a hosted authentication system.

Run and review

Install Node 20+ and Python 3.11+, run **npm ci --ignore-scripts**, then **npm start**. Open 127.0.0.1:4323. Use **npm test**, **npm run test:sdk** and **npm run test:contracts** for checks. Foundry is required for the contract suite. Setup and registry guides are in the repository.

Primary references

[Gluwa USC SDK 0.18.0 and official examples](#)
[Creditcoin native verifier migration guide](#)
[BUIDL CTC 2026 Fall requirements](#)